

Seleção, Iteração e Vetores em Java

Comandos Condicionais, Laços de Repetição, Vetores e Matrizes

Universidade Estadual da Paraíba
Linguagem de Programação II

September 22, 2026

O que vamos aprender hoje?

- ① Comandos de **Seleção** (if, else, switch)
- ② Comandos de **Iteração** (while, do-while, for)
- ③ **Vetores**: guardando várias informações do mesmo tipo
- ④ **Matrizes**: organizando dados em linhas e colunas

- 1 Comandos de Seleção
- 2 Comandos de Iteração
- 3 Vetores
- 4 Matrizes

Sumário

1 Comandos de Seleção

2 Comandos de Iteração

3 Vetores

4 Matrizes

Até agora...

- Nosso código sempre executa **as mesmas instruções**, uma após a outra, em sequência;
- Mas programas de verdade precisam **tomar decisões** – fazer coisas diferentes dependendo dos dados.

Para isso, existem os **comandos de seleção**.

O que é um Comando de Seleção?

😊 Definição: Seleção

Também chamado de **comando condicional**. Permite que o programa escolha um caminho entre dois ou mais, dependendo de uma **condição** ser verdadeira ou falsa.

Seleção: uma analogia

Analogia

Pense numa **bifurcação na estrada** com uma placa: se a condição for "vai pra praia", você segue por um caminho; caso contrário, segue por outro. O programa faz exatamente essa escolha, só que com código.

O Comando if

```
1 if (condicao) {  
2     // código executado se condicao for verdadeira  
3 }
```

- if: palavra-chave que inicia o comando;
- (condicao): uma expressão que resulta em true ou false;
- { ... }: o bloco que só executa se a condição for true.

Operadores Relacionais

Operador	Significado
>	maior que
<	menor que
>=	maior ou igual a
<=	menor ou igual a
==	igual a
!=	diferente de

Exemplo Prático

😊 Exemplo: Maior de idade

```
1 int idade = 20;  
2 if (idade >= 18) {  
3     System.out.println("Maior de idade");  
4 }
```

O Comando if-else

```
1 if (condicao) {  
2     // executa se condicao for verdadeira  
3 } else {  
4     // executa se condicao for falsa  
5 }
```

O else cobre o **outro** caminho – exatamente um dos dois blocos é executado, nunca os dois.

Exemplo Prático

😊 Exemplo: Par ou ímpar

```
1 int numero = 7;  
2 if (numero % 2 == 0) {  
3     System.out.println("Par");  
4 } else {  
5     System.out.println("Ímpar");  
6 }
```

Operadores Lógicos

Operador	Significado	Exemplo
<code>&&</code>	E (<i>and</i>) – as duas precisam ser verdadeiras	<code>a && b</code>
<code> </code>	OU (<i>or</i>) – pelo menos uma verdadeira	<code>a b</code>
<code>!</code>	NÃO (<i>not</i>) – inverte o valor	<code>!a</code>

Exemplo Prático

😊 Exemplo: Nota válida

```
1 double nota = 7.5;
2 if (nota >= 0 && nota <= 10) {
3     System.out.println("Nota válida");
4 } else {
5     System.out.println("Nota inválida");
6 }
```

Vários Caminhos: else if

Quando temos **mais de duas** opções, encadeamos com else if:

```
1 if (condicao1) {  
2     // ...  
3 } else if (condicao2) {  
4     // ...  
5 } else {  
6     // ...  
7 }
```

As condições são testadas **em ordem**, de cima para baixo; a primeira verdadeira "ganha".

Exemplo Prático

😊 Exemplo: Faixa etária

```
1  int idade = 15;
2  if (idade < 12) {
3      System.out.println("Crianca");
4  } else if (idade < 18) {
5      System.out.println("Adolescente");
6  } else if (idade < 60) {
7      System.out.println("Adulto");
8  } else {
9      System.out.println("Idoso");
10 }
```

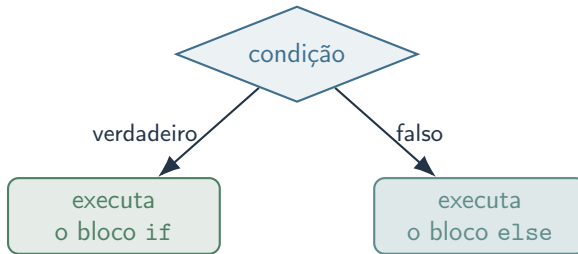
Erro Comum

```
1 if (idade = 18) { // errado!
```

☹ Erro Comum

= é o operador de **atribuição** (guarda um valor); == é o operador de **comparação** (verifica igualdade). Trocar um pelo outro é um dos erros mais comuns em quem está começando.

Diagrama: como o if-else decide



Por que outro comando de seleção?

- Às vezes comparamos a **mesma variável** com muitos valores possíveis;
- Uma cadeia longa de `else if` fica repetitiva e difícil de ler.

Para esses casos, o Java tem o comando **switch**.

O Comando switch

```
1 switch (variavel) {  
2     case valor1:  
3         // codigo  
4         break;  
5     case valor2:  
6         // codigo  
7         break;  
8     default:  
9         // codigo se nao bater nenhum caso  
0 }
```

Quebrando o switch

- `switch (variavel)`: a variável que será comparada;
- `case valor::` um dos valores possíveis para comparar;
- `break;`: encerra o switch assim que um caso bate;
- `default::` executado se **nenhum** case corresponder (opcional).

Exemplo Prático

☺ Exemplo: Dia da semana

```
1  int dia = 3;
2  switch (dia) {
3      case 1:
4          System.out.println("Domingo");
5          break;
6      case 2:
7          System.out.println("Segunda");
8          break;
9      case 3:
10         System.out.println("Terça");
11         break;
12     default:
13         System.out.println("Outro dia");
14 }
```

Erro Comum: esquecer o break

```
1 case 1:  
2     System.out.println("Domingo");  
3     // faltou o break!  
4 case 2:  
5     System.out.println("Segunda");  
6     break;
```

☹ Erro Comum

Sem o `break`, o Java continua executando os case **seguintes** também, mesmo que não batam – esse comportamento se chama *fall-through*. No exemplo, `dia = 1` imprimiria "Domingo" e "Segunda".

Atividade Prática – Seleção

Sistema de cobrança de estacionamento

Desenvolva um programa em Java que leia o **tipo do veículo**, a **quantidade de horas estacionadas** e informe o valor final a ser pago.

- Tipo 1 – **Moto**: R\$ 3,00 por hora;
- Tipo 2 – **Carro**: R\$ 5,00 por hora;
- Tipo 3 – **Caminhonete**: R\$ 8,00 por hora;
- Qualquer outro tipo deve ser considerado **inválido**.

Além disso:

- Até **2 horas**, não existe desconto;
- De **3 até 5 horas**, inclusive, aplique **10% de desconto**;
- Acima de **5 horas**, aplique **20% de desconto**.

Utilize obrigatoriamente `switch` e pelo menos uma estrutura `if / else`.

Resposta – Sistema de Estacionamento

```
1 import java.util.Scanner;
2
3 public class Estacionamento {
4
5     public static void main(String[] args) {
6
7         Scanner leitor =
8             new Scanner(System.in);
9
10        System.out.print(
11            "Tipo do veiculo: ");
12        int tipo = leitor.nextInt();
13
14        System.out.print(
15            "Quantidade de horas: ");
16        int horas = leitor.nextInt();
17
18        double total = 0;
19
20        switch (tipo) {
21            case 1:
22                total = horas * 3;
23                break;
24            case 2:
25                total = horas * 5;
26                break;
```

```
1         case 3:
2             total = horas * 8;
3             break;
4
5         default:
6             System.out.println(
7                 "Tipo invalido");
8         }
9
10        if (horas >= 3 &&
11            horas <= 5) {
12
13            total = total * 0.90;
14
15        } else if (horas > 5) {
16
17            total = total * 0.80;
18        }
19
20        if (total > 0) {
21            System.out.println(
22                "Valor final: R$ " + total);
23        }
24
25        leitor.close();
26    }
27 }
```

♡ Checkpoint

- `if`: executa um bloco só se a condição for verdadeira;
- `if-else`: escolhe entre dois caminhos;
- `else if`: encadeia várias condições, em ordem;
- `switch`: compara uma variável com vários valores possíveis – não esqueça o `break`!

Sumário

- 1 Comandos de Seleção
- 2 Comandos de Iteração**
- 3 Vetores
- 4 Matrizes

E se precisarmos repetir algo?

- Imagine ter que imprimir os números de 1 a 1000;
- Copiar e colar `System.out.println` mil vezes seria péssimo!

Para repetir tarefas, existem os **comandos de iteração** (ou **laços de repetição**).

O que é um Comando de Iteração?

😊 Definição: Iteração

É um comando que **repete** um bloco de código várias vezes, enquanto (ou até que) uma condição seja satisfeita.

Iteração: uma analogia

Analogia

Pense numa receita de bolo que diz: "bata a massa até ficar homogênea". Você repete o mesmo movimento – bater – e só para quando a condição ("ficar homogênea") é satisfeita. Isso é um laço de repetição!

O Comando while

```
1 while (condicao) {  
2     // repete enquanto condicao for verdadeira  
3 }
```

- O Java testa a condicao **antes** de cada repetição;
- Se a condição já começar falsa, o bloco **nunca** executa.

Exemplo Prático

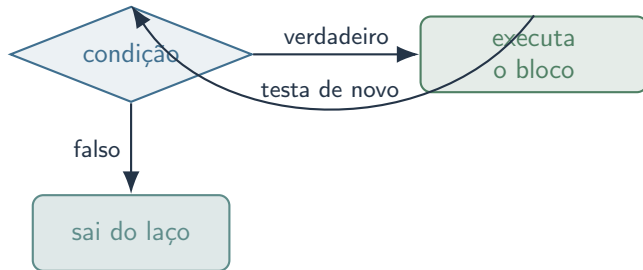
😊 Exemplo: Contando de 1 a 5

```
1 int contador = 1;
2 while (contador <= 5) {
3     System.out.println(contador);
4     contador++;
5 }
```

😊 Dica

Nunca esqueça de atualizar a variável (contador++) dentro do laço – é ela que, mais cedo ou mais tarde, torna a condição falsa.

Diagrama: como o while repete



O Comando do-while

```
1 do {  
2     // executa pelo menos uma vez  
3 } while (condicao);
```

- Repare: o bloco vem **antes** da condição;
- O Java testa a condicao **depois** de cada repetição;
- Por isso, o bloco sempre executa **pelo menos uma vez**, mesmo que a condição já comece falsa.

while x do-while

	while	do-while
Quando testa a condição	Antes	Depois
Executa pelo menos 1 vez?	Não necessariamente	Sempre

Exemplo Prático

😊 Exemplo: Menu que só sai com "sair"

```
1 Scanner leitor = new Scanner(System.in);  
2 String opcao;  
3 do {  
4     System.out.print("Digite 'sair' para encerrar: ");  
5     opcao = leitor.nextLine();  
6 } while (!opcao.equals("sair"));
```

Por que outro comando de repetição?

- Muitas vezes já sabemos **exatamente quantas vezes** repetir;
- Escrever isso com `while` funciona, mas exige 3 linhas separadas (iniciar, testar, atualizar).

O comando `for` junta essas 3 partes numa linha só.

O Comando for

```
1 for (inicializacao; condicao; incremento) {  
2     // repete enquanto condicao for verdadeira  
3 }
```

Vamos entender cada uma das 3 partes, com calma.

Parte 1: inicialização

```
1 for (int i = 1; i <= 10; i++) {
```

- `int i = 1`: cria a variável de controle e dá seu valor inicial;
- Executada **uma única vez**, antes de tudo começar.

Parte 2: condição

```
1 for (int i = 1; i <= 10; i++) {
```

- `i <= 10`: testada **antes** de cada repetição, igual ao `while`;
- Enquanto `for` `true`, o laço continua.

Parte 3: incremento

```
1 for (int i = 1; i <= 10; i++) {
```

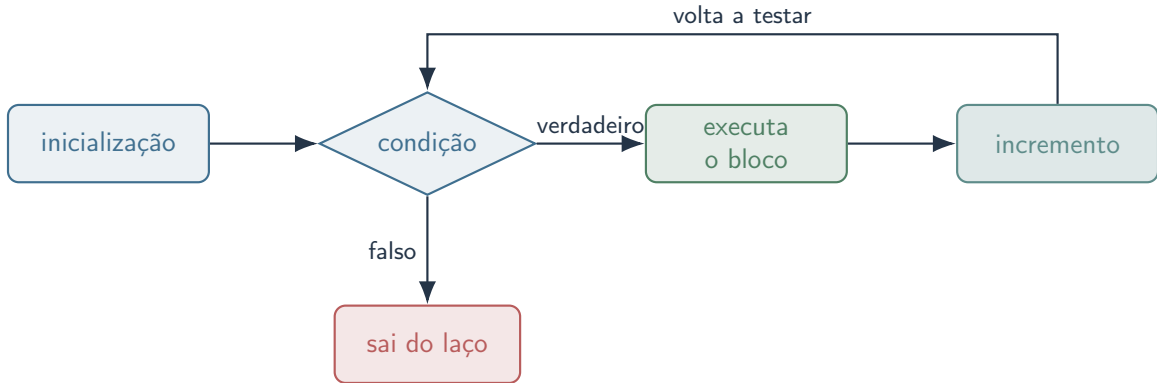
- `i++`: executado ao **final de cada repetição**;
- É o que garante que, cedo ou tarde, a condição vire falsa.

Exemplo Prático

😊 Exemplo: Somando de 1 a 10

```
1 int soma = 0;
2 for (int i = 1; i <= 10; i++) {
3     soma = soma + i;
4 }
5 System.out.println("Soma: " + soma);
```

Diagrama: como o for repete



Laços Aninhados

- Um `for` (ou `while`) pode conter **outro** laço dentro dele;
- Para cada repetição do laço de fora, o laço de dentro roda **completo**;
- Muito útil para trabalhar com padrões, tabelas e, como veremos, **matrizes**.

Exemplo Prático

☺ Exemplo: Tabuada

```
1 for (int i = 1; i <= 5; i++) {  
2     for (int j = 1; j <= 10; j++) {  
3         System.out.println(i + " x " + j + " = " + (i * j));  
4     }  
5 }
```

Para cada valor de i (1 a 5), o laço de dentro percorre j inteiro (1 a 10).

O Comando break

```
1 for (int i = 1; i <= 10; i++) {  
2     if (i == 5) {  
3         break;  
4     }  
5     System.out.println(i);  
6 }
```

break encerra o laço **imediatamente**, mesmo que a condição ainda fosse verdadeira. Esse exemplo imprime só 1, 2, 3, 4.

O Comando continue

```
1 for (int i = 1; i <= 10; i++) {  
2     if (i % 2 == 0) {  
3         continue;  
4     }  
5     System.out.println(i);  
6 }
```

continue pula só a **repetição atual** e segue para a próxima. Esse exemplo imprime apenas os números ímpares.

Erro Comum: Laço Infinito

☹ Erro Comum

Esquecer de atualizar a variável de controle – por exemplo, um `while` cuja condição nunca fica falsa porque ninguém faz `contador++` lá dentro. O programa trava, repetindo para sempre.

Erro Comum: Off-by-one

```
1 for (int i = 0; i <= 5; i++) { // sera que era isso mesmo?
```

☹ Erro Comum

Confundir `<` com `<=` (ou o valor inicial) faz o laço repetir uma vez **a mais** ou **a menos** do que o planejado. Sempre confira: quantas vezes eu **realmente** quero repetir?

Atividade Prática – Laços de Repetição

Identificando números primos

Desenvolva um programa em Java que leia números inteiros positivos **até que o valor 0 seja informado**.

A cada número digitado, o programa deve indicar imediatamente se ele é **primo** ou **não primo**.

Um número é primo quando possui exatamente dois divisores: **1 e ele mesmo**.

Exemplo:

Entrada	Saída
7	7 é primo
10	10 não é primo
13	13 é primo
2	2 é primo
0	programa encerrado

Resposta – Identificando Números Primos

```
1 import java.util.Scanner;
2
3 public class NumerosPrimos {
4
5     public static void main(String[] args) {
6
7         Scanner leitor =
8             new Scanner(System.in);
9
10        System.out.print(
11            "Digite um numero: ");
12        int numero = leitor.nextInt();
13
14        while (numero != 0) {
15
16            int divisores = 0;
17
18            for (int i = 1;
19                i <= numero; i++) {
20
21                if (numero % i == 0) {
22                    divisores++;
23                }
24            }
25        }
26    }
27 }
```

```
1         if (divisores == 2) {
2             System.out.println(
3                 numero + " e primo");
4         } else {
5             System.out.println(
6                 numero + " nao e primo");
7         }
8
9        System.out.print(
10            "Digite um numero: ");
11        numero = leitor.nextInt();
12    }
13
14    System.out.println(
15        "Programa encerrado");
16
17    leitor.close();
18 }
19 }
```

Atividade Prática – Laços de Repetição

Acesso ao sistema

Escreva um programa em Java que solicite uma senha ao usuário.
A senha correta é 1234.

Enquanto a senha informada estiver incorreta, o programa deve solicitar uma nova tentativa.
Quando a senha correta for digitada, deve exibir:

Acesso permitido

A senha deve ser solicitada **pelo menos uma vez**, e a decisão de repetir deve ocorrer somente após a tentativa realizada.

Resposta – Acesso ao Sistema

```
1 import java.util.Scanner;
2
3 public class Acesso {
4
5     public static void main(String[] args) {
6
7         Scanner leitor = new Scanner(System.in);
8
9         int senha;
10
11         do {
12             System.out.print("Digite a senha: ");
13             senha = leitor.nextInt();
14
15         } while (senha != 1234);
16
17         System.out.println("Acesso permitido");
18
19         leitor.close();
20     }
21 }
```

♡ Checkpoint

- while: testa antes – pode nunca executar;
- do-while: testa depois – executa pelo menos uma vez;
- for: ideal quando já sabemos quantas repetições;
- break sai do laço; continue pula para a próxima repetição.

Sumário

- 1 Comandos de Seleção
- 2 Comandos de Iteração
- 3 Vetores**
- 4 Matrizes

E se precisarmos guardar muitos valores?

- Imagine guardar as notas de 40 alunos;
- Criar 40 variáveis (nota1, nota2, ..., nota40) seria inviável!

Para isso, existem os **vetores**.

O que é um Vetor?

😊 Definição: Vetor

Também chamado de *array*. É uma coleção de valores do **mesmo tipo**, guardados em posições sequenciais, cada uma identificada por um **índice**.

Vetor: uma analogia

Analogia

Pense numa fileira de **escaninhos numerados**, como os de uma agência de correios. Cada escaninho guarda uma coisa, e você acessa cada um pelo seu número – não precisa dar um nome diferente para cada escaninho.

Declarando um Vetor

```
1 int[] notas = new int[5];
```

- `int[]`: o tipo dos elementos, seguido de colchetes – é um vetor **de** `int`;
- `notas`: o nome da variável;
- `new int[5]`: cria o vetor de fato, reservando espaço para **5** elementos.

Os Índices Começam em ZERO!

Atenção: em Java, o primeiro elemento de um vetor está sempre no índice 0!

Um vetor de tamanho 5 tem índices 0, 1, 2, 3, 4 – **nunca** o índice 5.

Visualizando um Vetor



```
int[] numeros = {10, 20, 30, 40, 50};
```

Acessando e Alterando Elementos

```
1 int[] notas = new int[5];  
2 notas[0] = 10;  
3 notas[1] = 8;  
4 System.out.println(notas[0]);
```

- `notas[0] = 10`: guarda o valor 10 na posição 0;
- `notas[0]`: lê o valor guardado na posição 0.

Percorrendo um Vetor

😊 Exemplo: Imprimindo todas as notas

```
1 int[] notas = {7, 8, 5, 9, 6};  
2 for (int i = 0; i < notas.length; i++) {  
3     System.out.println(notas[i]);  
4 }
```

Esse padrão – for de 0 até length, usando i como índice – é **extremamente** comum em Java.

O Atributo `.length`

```
1 System.out.println(notas.length);
```

- `.length` devolve o **tamanho** do vetor;
- É um **atributo**, não um método – repare que não tem parênteses!

☹ Erro Comum

Cuidado: `String` usa `.length()` **com** parênteses (é um método). `Vetor` usa `.length` **sem** parênteses (é um atributo). Confundir os dois é um erro clássico.

Exemplo Prático

😊 Exemplo: Somando os elementos

```
1 int[] numeros = {10, 20, 30, 40, 50};
2 int soma = 0;
3 for (int i = 0; i < numeros.length; i++) {
4     soma = soma + numeros[i];
5 }
6 System.out.println("Soma: " + soma);
```

Exemplo Prático

😊 Exemplo: Encontrando o maior valor

```
1 int[] numeros = {10, 45, 23, 67, 12};
2 int maior = numeros[0];
3 for (int i = 1; i < numeros.length; i++) {
4     if (numeros[i] > maior) {
5         maior = numeros[i];
6     }
7 }
8 System.out.println("Maior: " + maior);
```

Começamos supondo que o **primeiro** elemento é o maior, e vamos comparando com os demais.

Vetor com Valores Iniciais

```
1 int[] numeros = {10, 20, 30, 40, 50};
```

- Podemos declarar e já preencher o vetor numa linha só;
- O tamanho é definido automaticamente pela quantidade de valores entre chaves.

Erro Comum

```
1 int[] numeros = new int[5];  
2 System.out.println(numeros[5]); // erro!
```

☹ Erro Comum

`ArrayIndexOutOfBoundsException`: tentar acessar um índice que não existe no vetor. Se o tamanho é 5, os índices válidos vão de 0 a 4 – o índice 5 está **fora** dos limites.

Atividade Prática – Vetores

Soma dos dois maiores valores

Desenvolva um programa em Java que solicite ao usuário a **quantidade de posições** de um vetor de números inteiros.

Considere que o tamanho informado será sempre **maior ou igual a 2**.

Em seguida, o programa deve criar o vetor com o tamanho informado e solicitar o preenchimento de todas as suas posições.

Depois do preenchimento, o programa deve percorrer o vetor, encontrar os **dois maiores valores armazenados** e exibir a **soma entre eles**.

Por exemplo, para o vetor:

8, 3, 10, 4, 7

os dois maiores valores são 10 e 8. Portanto, o programa deverá exibir:

Resposta – Soma dos Maiores Valores

```
1 import java.util.Scanner;
2
3 public class MaioresValores {
4
5     public static void main(String[] args) {
6
7         Scanner leitor =
8             new Scanner(System.in);
9
10        System.out.print(
11            "Tamanho do vetor: ");
12        int tamanho = leitor.nextInt();
13
14        int[] valores =
15            new int[tamanho];
16
17        for (int i = 0;
18            i < valores.length; i++) {
19
20            System.out.print(
21                "Valor: ");
22
23            valores[i] =
24                leitor.nextInt();
25        }
26
27        int maior;
28        int segundoMaior;
```

```
1     if (valores[0] > valores[1]) {
2         maior = valores[0];
3         segundoMaior = valores[1];
4     } else {
5         maior = valores[1];
6         segundoMaior = valores[0];
7     }
8
9     for (int i = 2;
10        i < valores.length; i++) {
11
12        if (valores[i] > maior) {
13            segundoMaior = maior;
14            maior = valores[i];
15
16        } else if (valores[i] >
17            segundoMaior) {
18
19            segundoMaior = valores[i];
20        }
21    }
22
23    System.out.println(
24        "Soma: " +
25        (maior + segundoMaior));
26
27    leitor.close();
28 }
```

♡ Checkpoint

- Vetor: coleção de valores do mesmo tipo, acessados por índice;
- Índices sempre começam em 0 e vão até `length - 1`;
- `.length` (sem parênteses) devolve o tamanho;
- Percorrer um vetor: `for (int i = 0; i < v.length; i++)`.

Sumário

- 1 Comandos de Seleção
- 2 Comandos de Iteração
- 3 Vetores
- 4 Matrizes**

E se os dados tiverem linhas e colunas?

- Um vetor guarda uma **fileira única** de valores;
- Mas e um tabuleiro de jogo da velha? Uma planilha? Um mapa de pixels?

Para dados organizados em linhas e colunas, existem as **matrizes**.

O que é uma Matriz?

😊 Definição: Matriz

É, na prática, um **vetor de vetores**: organiza os dados em **linhas** e **colunas**, formando uma grade.

Matriz: uma analogia

☺ Analogia

Pense numa planilha do Excel, ou num tabuleiro de jogo da velha: cada informação mora numa **linha** e numa **coluna** específicas – não basta um número, precisamos de **dois**.

Declarando uma Matriz

```
1 int [][] tabuleiro = new int [3] [3];
```

- `int [] []`: dois pares de colchetes – um vetor de vetores;
- `new int [3] [3]`: cria uma matriz com **3 linhas** e **3 colunas**.

Visualizando uma Matriz

	coluna 0	coluna 1	coluna 2
linha 0	[0][0]	[0][1]	[0][2]
linha 1	[1][0]	[1][1]	[1][2]
linha 2	[2][0]	[2][1]	[2][2]

Acessando Elementos

```
1 tabuleiro[0][0] = 1;  
2 tabuleiro[1][2] = 5;  
3 System.out.println(tabuleiro[0][0]);
```

- O **primeiro** colchete é a **linha**;
- O **segundo** colchete é a **coluna**.

Exemplo Prático

☺ Exemplo: Tabuleiro da velha vazio

```
1 char[][] tabuleiro = new char[3][3];
2 for (int linha = 0; linha < 3; linha++) {
3     for (int coluna = 0; coluna < 3; coluna++) {
4         tabuleiro[linha][coluna] = '-';
5     }
6 }
```

Percorrendo uma Matriz

- Para visitar **todas** as posições, usamos **dois** for aninhados;
- O laço de **fora** percorre as linhas;
- O laço de **dentro** percorre as colunas **daquela** linha.

Exemplo Prático

😊 Exemplo: Preenchendo e imprimindo

```
1 int[][] matriz = new int[2][3];
2 for (int i = 0; i < matriz.length; i++) {
3     for (int j = 0; j < matriz[i].length; j++) {
4         matriz[i][j] = i + j;
5         System.out.print(matriz[i][j] + " ");
6     }
7     System.out.println();
8 }
```

Exemplo Prático

☺ Exemplo: Somando todos os elementos

```
1 int[][] matriz = {{1,2,3},{4,5,6},{7,8,9}};  
2 int soma = 0;  
3 for (int i = 0; i < matriz.length; i++) {  
4     for (int j = 0; j < matriz[i].length; j++) {  
5         soma = soma + matriz[i][j];  
6     }  
7 }  
8 System.out.println("Soma: " + soma);
```

☹ Erro Comum

Trocar a ordem de linha e coluna – escrever `matriz[coluna][linha]` em vez de `matriz[linha][coluna]`. Em matrizes bem grandes, isso costuma gerar um `ArrayIndexOutOfBoundsException` bem confuso de encontrar.

♡ Checkpoint

- Matriz: um vetor de vetores, organizado em linhas e colunas;
- `matriz[linha][coluna]` acessa um elemento específico;
- Para percorrer tudo, usamos dois `for` aninhados: linha por fora, coluna por dentro.

- ❶ `if`, `if-else` e `else if` escolhem **qual** caminho seguir; `switch` compara uma variável com vários valores;
- ❷ `while` testa antes, `do-while` testa depois, `for` é ideal quando já sabemos quantas repetições;
- ❸ **Vetores** guardam várias informações do mesmo tipo, acessadas por índice (começando em 0);
- ❹ **Matrizes** organizam dados em linhas e colunas – um vetor de vetores.

Mãos à Obra!